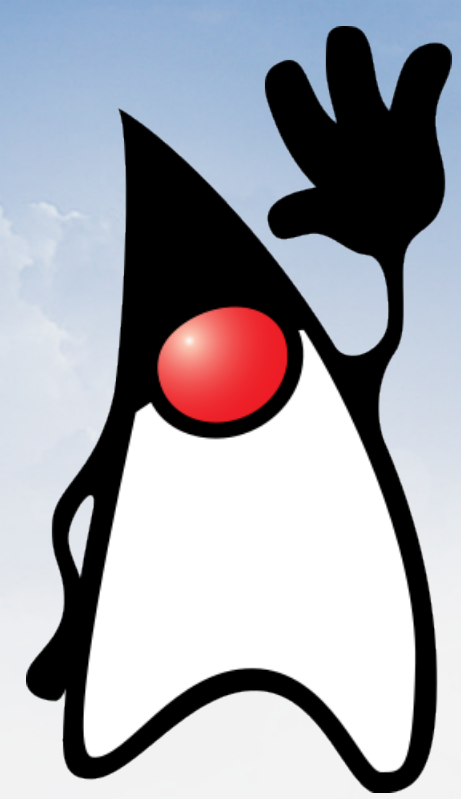




Linked Lists

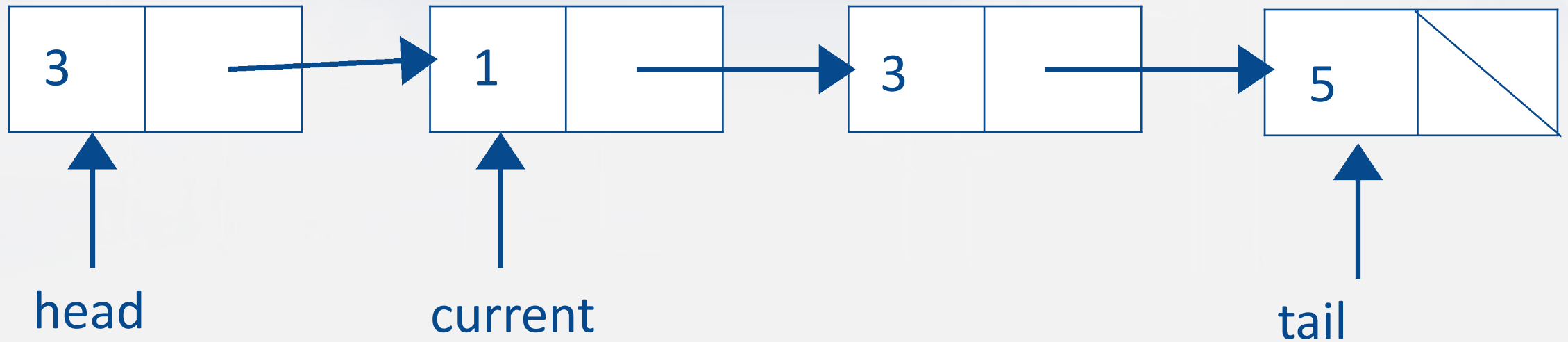
COP 3330
Department of Computer Science
University of Central Florida
Dr. Bacanli

Linked List

- Set of items/nodes linked together
 - Single linked list (only store the next item reference)
 - Doubly linked list (store next and previous item reference)
- Reaching an element worst case: $O(n)$
- When we reach, delete, add, update : $O(1)$
- Used as a basis to implement other data structures
 - Queue, Stack, Hash(kind of)
- The hard disk memory in computers work similar with
LinkedLists

Linked list

- Elements linked together



Important notes

- In singly linked list we do not have back reference
 - We can not reach an item if we passed it
- In singly linked list, the order of the operations is important
 - Our only connection is one reference
 - The code will not work although the commands are correct
- Doubly linked list is more forgiving for order of operations

```
package allLinkedList;
public class Node {
    private int item;
    private Node next;

    public Node(int el) {
        item=el;
        next=null;//initially next is null
    }

    public void setNext(Node gnext) {
        next=gnext;
    }

    public Node getNext() {
        return next;
    }

    public int getItem() {
        return item;
    }

    public String toString() {
        return "Node item is: "+item;
    }
}
} //end of Node class
```

Node

```
private void setItem(int it){
    item=it;
}
// Do NOT implement this
method in your assignment.
```

```
package allLinkedList;
public class LinkedList {
    private Node head;

    public LinkedList() {
        head=null;
    }

    private Node getHead() {
        return head;
    }

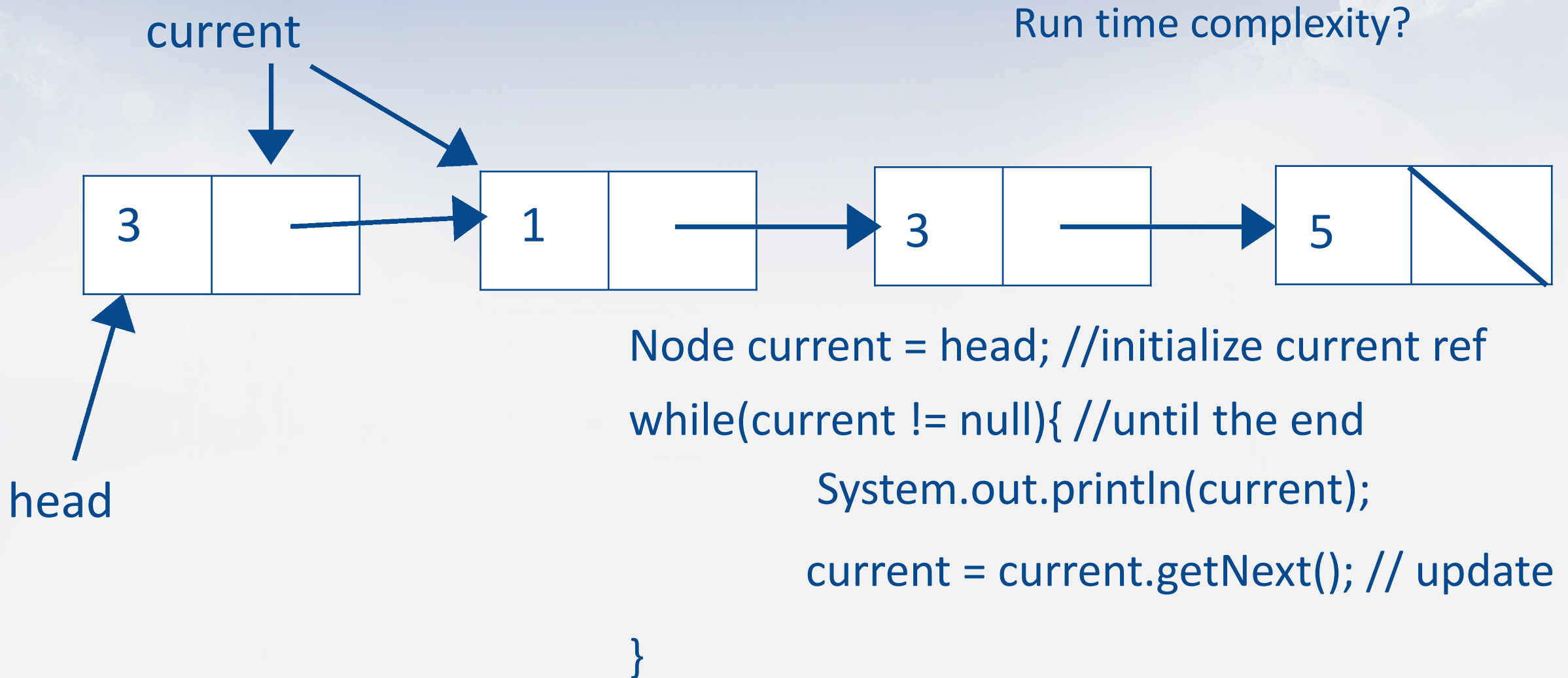
    public boolean isEmpty() {
        return head == null;
    }
}
```

Linked List

In our examples, there won't be a dummy head node!

We will hold the reference of the head not the head node itself!

Traversing Linked list



What to do if we want to add to the head/tail?

- It is better to modify the references and the item of the newly added node first!
- We can leave the important modifications later.

Adding a node to head

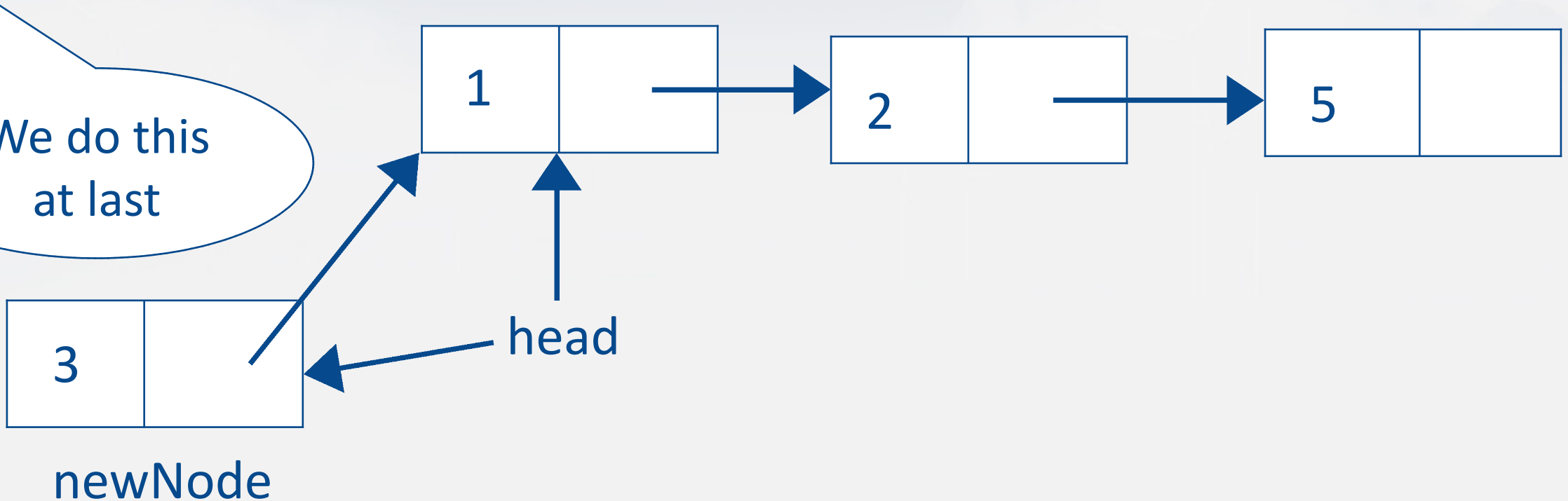
```
Node newNode = new Node(3);
```

```
newNode.setNext(head);
```

```
head = newNode;
```

Run time complexity?

We do this
at last



Add a node to tail

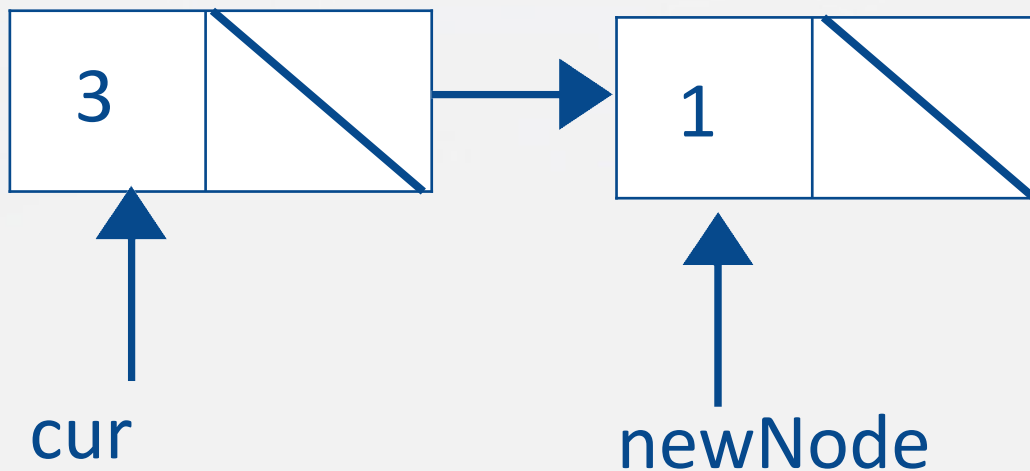
Run time complexity?

We always need to check if the list is empty if we don't, then the while loop will give ERROR

For an empty list:

cur(head) is null therefore cur.getNext() is undefined.

Can we get away from checking isEmpty?



```
Node newNode = new Node(1);
```

```
If(the list is empty) then
```

```
    head=newNode
```

```
else
```

```
    Node cur=head;
```

```
    while(cur.getNext() != null) {  
        cur=cur.getNext();
```

```
    }
```

```
// Now we reached the last/tail node  
// newNode's next is null by default
```

```
cur.setNext(newNode);
```

What to do if we want to delete the head/tail?

In C/C++ we will need to cut off any references **to** and *preferably* **from** the Node object

Then we will need to delete the Node object

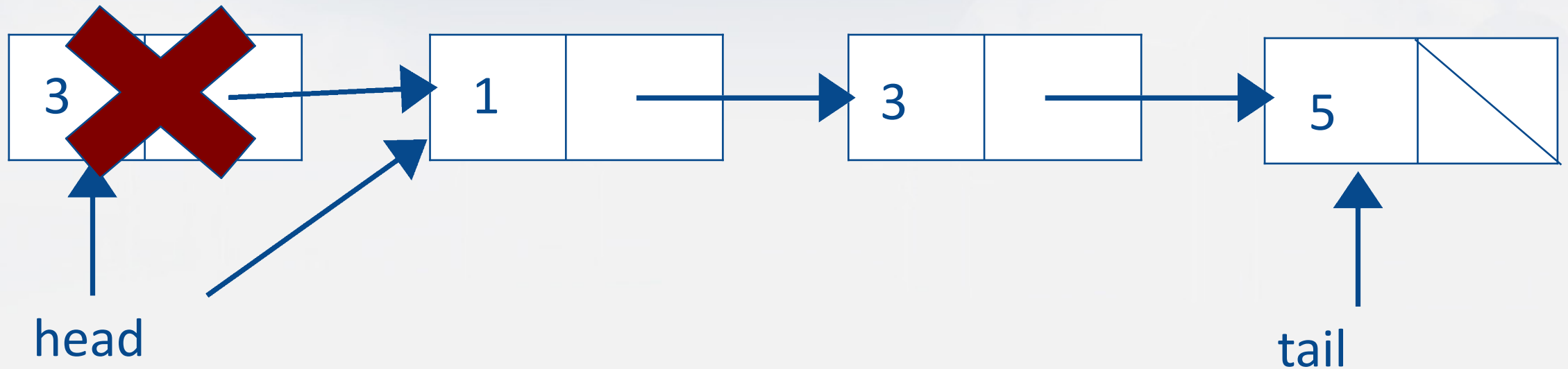
In Java, cutting off the references **to** that Node is sufficient.
If nothing is pointing to that Node, it will be unreachable.

What happens if an object is unreachable?

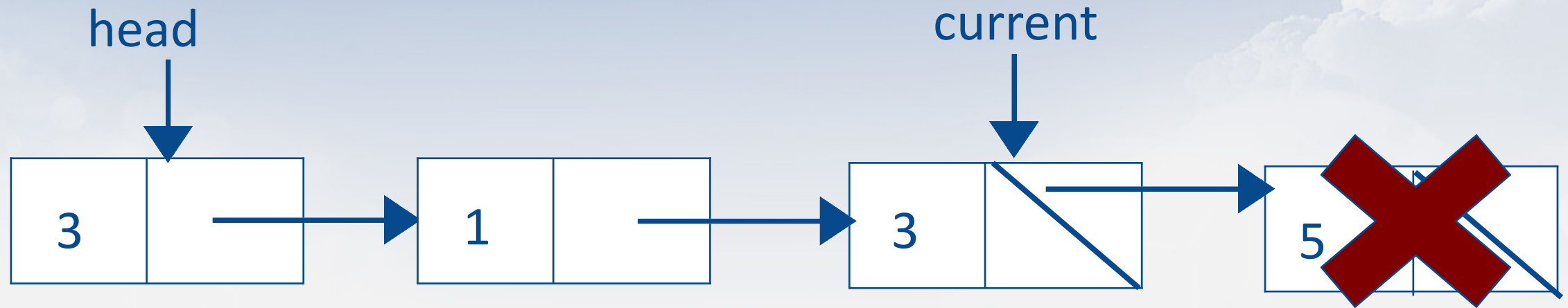
Deleting the head in Linked list

`head = head.getNext();`

Can you delete the tail that easily also?



Deleting the tail in Linked list



```
Node current = head; //initialize current
while( current.getNext().getNext() !=null ){ // reach until the tail
    current = current.getNext(); // update current
}
// now current is before the tail node! current.next.next is null
current.setNext(null);
```

Run time complexity?

What to do if we want to add/delete from middle?

We may need to check if we are deleting the head or adding before the head!

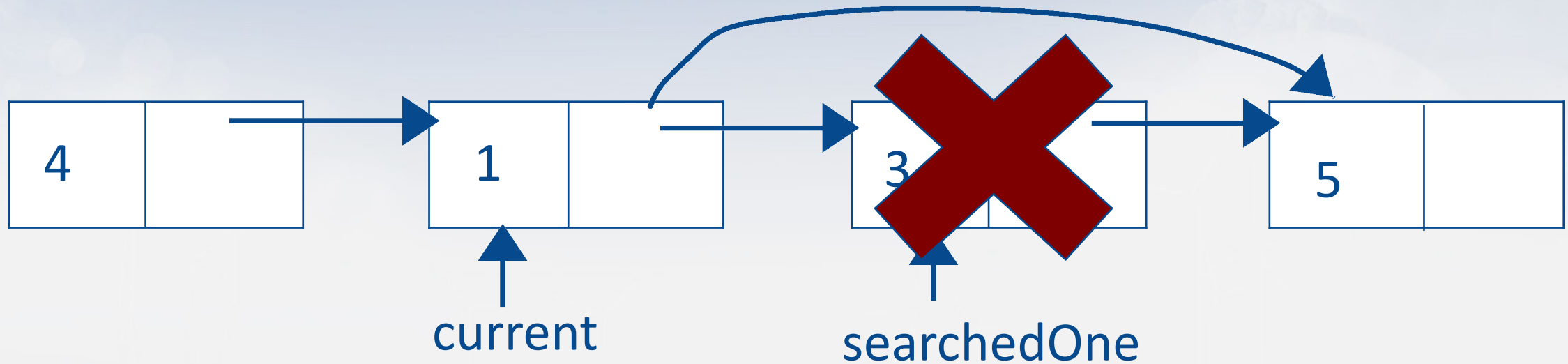
In deletion process, making the object unreachable is enough.

Also in the deletion process we need to reach the node right before the node we want to remove. If not?

The general keyword for removal is:
`cur.setNext(cur.getNext().getNext())`

Deleting from middle of Linked list

Also, We will need to check if head is null because *current.getNext()* will give an error



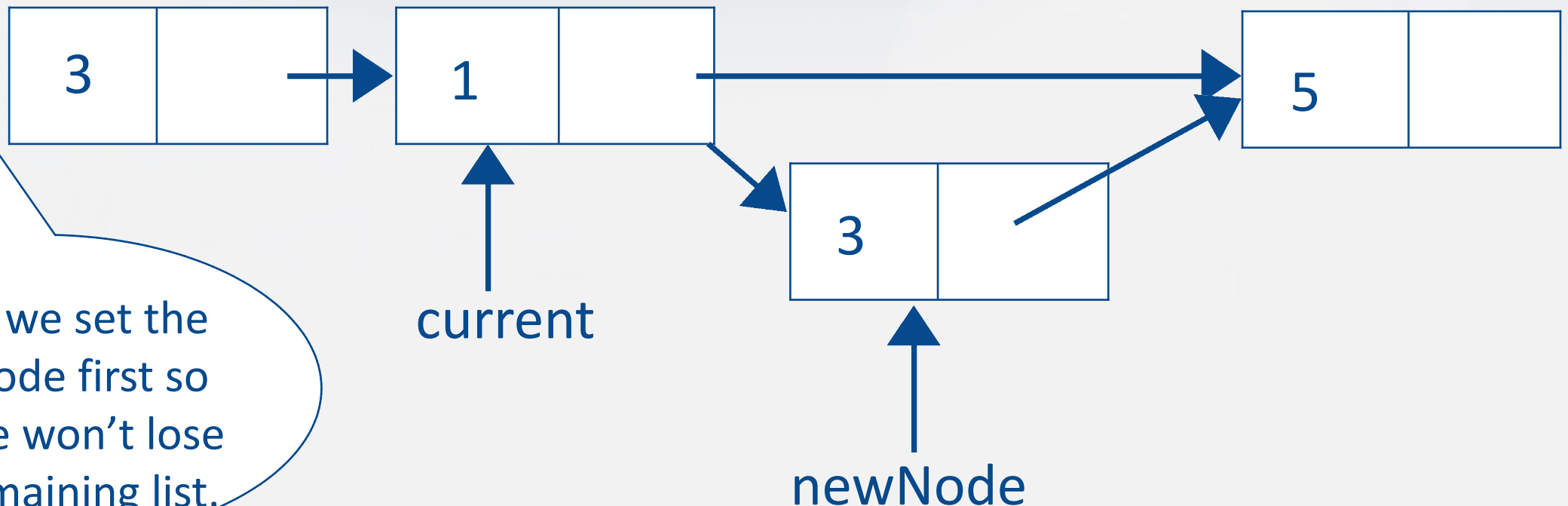
```
Node current=head; // we will go to the node right before our searched Elem
while(current.getNext() != null && current.getNext().getItem() != searchedElem) {
    current=current.getNext(); // we need to find item right before searchedOne
}
```

```
current.setNext( current.getNext().getNext() );
```

Adding a node to middle of Linked list

```
Node newNode = new Node(3);  
newNode.setNext(current.getNext());  
current.setNext(newNode);
```

Considering we want to add right after 1. We have reached the Node, which contains 1 as precondition, with *current*.



again, we set the newNode first so that we won't lose the remaining list.